

Object-Oriented Analysis and Design HW5

資電三 112504009 林紘喬

A. 設計問題

1. 一個 class 混了太多概念：

Customer 本來應該只表示顧客本身的資料，例如：

- 顧客編號
- 地址

但是這個 class 裡面還直接放了：

ProductID[100]

ProductCompanyID[100]

也就是把「顧客資料」和「購買紀錄」混在一起。

2. 這裡用了 parallel arrays 易出錯：

- ProductID[100]
- ProductCompanyID[100]
- used[100]

這三個陣列其實必須靠相同 index 對應，例如：

- ProductID[0]
- ProductCompanyID[0]
- used[0]

表示同一筆資料。

3. used[100] 是很不好的設計：

used[100] 的存在代表：

這個陣列中有些格子有用，有些格子沒用，要額外記錄，且不夠彈性

4. 維護困難：

之後如果想增加新需求，例如：

- 購買日期

- 購買數量
- 單價
- 折扣
- 訂單編號

那就會變成還要再加很多平行陣列，例如：

- PurchaseDate[100]
- Quantity[100]
- Discount[100]

整個 class 會越來越亂，也更容易出錯。

5. 無法清楚表達產品本身的資訊：

產品通常還會有：

- 產品名稱
- 價格
- 類別
- 公司名稱

如果只放 ID，很多資訊還是得另外找，而且目前設計也沒有清楚表達：

- Product 是不是一個獨立物件
- Company 是不是一個獨立物件

B. 改寫方式

- Customer class 只負責顧客本身的資料。
- Product class 只負責產品資料。
- Purchase class 用來表示某位顧客購買某個產品的關係。

```
#include <string>
```

```
#include <vector>
```

```
using namespace std;
```

```
class Product {  
public:  
    int productID;  
    int companyID;  
};  
class Purchase {  
public:  
    int customerID;  
    int productID;  
};  
class Customer {  
public:  
    int id;  
    string address;  
};
```